SYSTEMS · CONCURRENCY · PERFORMANCE ENGINEERING

Font rasterization concurrency in golang.org/x/image: experimental evidence, ownership, and a decision not to change upstream

Luiz Felipe Cruz Borges

Abstract

Sharing an `opentype.Face` across goroutines produces **data races** reported by the Go race detector. Two interpretations are possible: treat the sharing as a defect to fix with a patch, or as use incompatible with the documented contract that `font.Face` is not safe for concurrent use.

In this work, we distinguish those interpretations through mutable-state mapping, causal reproduction with 13 hypotheses and orthogonal controls, a performance baseline, and a comparison of six ownership patterns (A–F). We observe real concurrent accesses under sharing and, without `-race`, observable corruption (digest mismatch) and panics. The **one Face per worker** pattern (C) satisfies the contract, remains race-free, and on the `Glyph` path was statistically indistinguishable from E and F in our experiments (`benchstat`, ~), while A, B, and D were significantly slower. Among the requirements and alternatives evaluated, the only missing capability — independent mask lifetime — would require a public API change. We found no evidence that justifies an upstream change under those requirements; the expected functional diff remains empty.

Keywords: Go · data race · OpenType · ownership · API compatibility · benchstat · negative result

1. Motivation

Text and UI applications in Go often rasterize glyphs in parallel. Reusing a single `opentype.Face` reduces the apparent construction cost, but under `-race` the detector reports warnings; in some cases we also observed incorrect digests or panics.

If the reading is…	Then the coherent action is…
Implicit concurrent contract broken	Patch (internal mutex, redesign, new API)
Non-concurrent contract upheld	Ownership discipline; proposal only if demand exists

The evidence in this study favors the second line. We do not merely restate the documentation: we characterize which mutable states make concurrent use invalid, link those states to observable corruption, formalize an ownership model, and evaluate whether any upstream architectural change is justified by the requirements considered.

1.1 Research questions

ID	Question
RQ1 — Safety	Which mutable states on the font rasterization path require exclusive ownership under concurrent execution?
RQ2 — Correctness	Are these races merely detector artifacts, or do they manifest as corruption/panic in normal execution?
RQ3 — Performance	Which ownership pattern provides the best throughput/latency under different levels of parallelism?
RQ4 — API	Among the requirements and alternatives evaluated, is there a relevant capability that cannot be obtained with the current public API?

1.2 Contributions

ID	Contribution
C1 — Characterization	Characterization of the mutable state responsible for concurrency violations in opentype.Face and adjacent states on the drawing path.
C2 — Causal evidence	Target/control experiments that connect shared state to races and to observable corruption (digest mismatch / panic).
C3 — Ownership model	A **share / scratch / borrow** model that explains the concurrency contract without assuming an ownership type system.
C4 — Comparative evaluation	Experimental comparison of six ownership strategies under increasing concurrency, with statistical analysis.
C5 — Negative engineering result	Evidence that, under the evaluated requirements, no upstream implementation change is justified.

2. Ownership model in the current API

2.1 Documented position in the Go ecosystem

The `font.Face` interface in `golang.org/x/image/font` explicitly states that it is **not thread-safe**. The package's normative comment asserts:

> *A Face is not safe for concurrent use by multiple goroutines, as its methods may re-use implementation-specific caches and mask image buffers.* — `font.Face`, package `golang.org/x/image/font` (`font/font.go`)

That wording entered the accepted history with CL 13463 (**shiny/font**, 2015; local post-migration commit `eecb4e626fdb…`); the current `git blame` of `font/font.go` attributes the equivalent lines to that commit, with no later conceptual change [1].

The rationale is not an arbitrary ban on parallelism: the contract authorizes implementations to reuse caches and mask buffers across calls. The `Glyph` method reinforces the short lifetime of the bitmap:

> *The contents of the mask image returned by one Glyph call may change after the next Glyph call. Callers that want to cache the mask must make a copy.*

In contrast, `sfnt.Font` documents the correct concurrent path: the font's methods **are** safe across goroutines as long as each call uses a distinct `*sfnt.Buffer` (or `nil`). The Go platform already separates **shareable typographic data** from **exclusive rasterization state**. The high-level Face (`opentype.Face`) embeds that state and therefore inherits the non-concurrent contract.

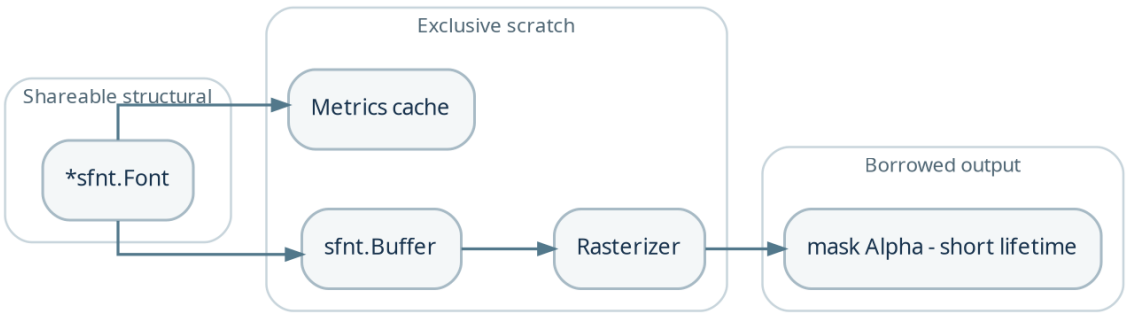
Parallelism remains allowed; what the contract forbids is **sharing the same Face instance**.

2.2 Three ownership concepts

We formalize three distinct roles (informal analogues of **share** / **mutate** / **borrow**, without a type system such as Rust [20]):

Role	Definition	Examples in the API
Shareable structural state	Data published after parse; concurrent reads valid under the documented premises	*sfnt.Font from immutable []byte; stable configuration (scale, hinting)
Exclusive mutable scratch state	Buffers and caches reused across calls; exclusive ownership per operation or worker	Metrics cache, sfnt.Buffer, vector.Rasterizer, header/backing of f.mask
Borrowed output	Returned value whose lifetime ends at the next call that reuses the same scratch	image.Alpha from Glyph (&f.mask); sfnt.Segments bound to the Buffer

Figure 1. *What may be shared vs what must be exclusive.*



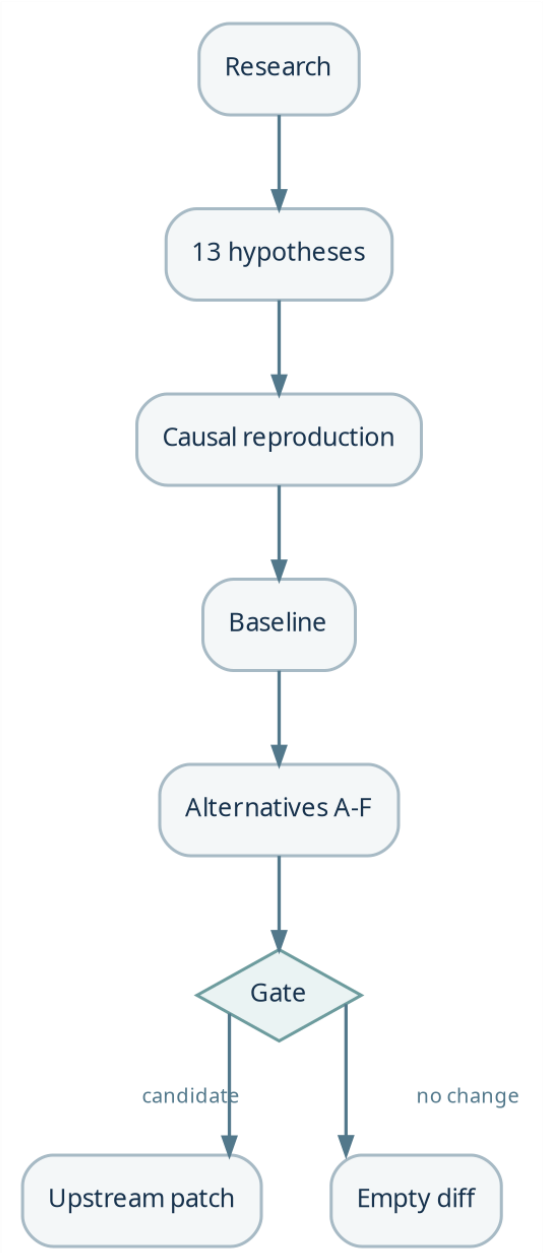
2.3 Change classification and the decision not to change upstream

For external readers we use descriptive labels (the internal harness uses a G0–G3 grid; **G1** = ambiguous behavioral change that requires discussion before implementation):

Term	Meaning
Internal change	No new public promise; compatible with valid sequential programs
Public API change	New exported surface, signature, or normative concurrency/lifetime promise — requires an issue/proposal in the Go project process [2]
Incompatible change	Breaks source, requires changes by external implementers, or weakens the contract
No upstream change	No change is authorized; expected functional diff = empty

3. Method

Figure 2. *Experimental pipeline; the lower branch is the actual path.*



3.1 System under test

Item	Value
Module	golang.org/x/image v0.45.0
Commit / tree	3ebddc7c… / 8840b964…
Packages under analysis	font, font/opentype, font/sfnt, vector
Focal APIs	font.Face, opentype.Face / NewFace, sfnt.Font / Buffer, font.Drawer, Glyph → image.Alpha
Typographic fixture	golang.org/x/image/font/gofont/goregular (TTF embedded in the module)
FaceOptions	Size 12, DPI 72, HintingNone

Upstream code remains mounted read-only; prototypes and reproducers live in an isolated harness module with `replace` to `/src`.

3.2 Experimental environment

Item	Value
Host	Windows 10 + Docker Desktop 4.77 / engine 29.5.3
Engine kernel	5.15.167.4-microsoft-standard-WSL2
Host CPU (model reported in benches)	Intel Core i5-9300H @ 2.40 GHz
nproc observed in the container	8 (host/WSL logical)
Container quota	--cpus 4 → cgroup v1 cpu.cfs_quota_us=400000 / period=100000 (**4 logical CPUs** available to the quota, not dedicated "physical CPUs")
Memory / PIDs	4 GiB (memory=memory-swap), 512 PIDs
Network	none (loopback only)
Image	x-image-font-harness:go1.25.0-v2, digest sha256:341bc773…
Go in the container	go1.25.0 linux/amd64, CGO_ENABLED=1, GOAMD64=v1, GOMAXPROCS=4 (primary matrix; also 1 and 2 for scalability)
Runner	versioned harness script (isolated container)

We do not claim physical topology pinning or cross-machine measurement; RQ3 interprets scalability against the **parallelism quota available to the process**.

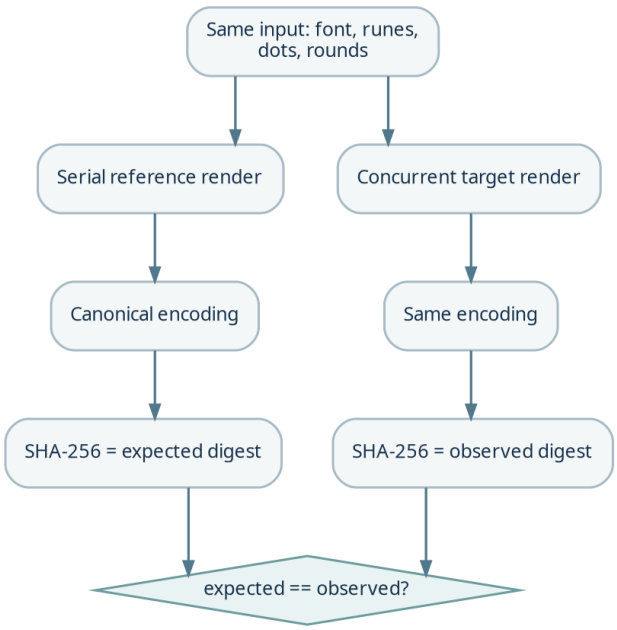
3.3 Race experiment

For each of the 13 causal IDs, the **target** shares exactly domain X and the **control** isolates X while keeping the rest identical. We use two workers, per-round barriers, and private sinks. The confirmation criterion is a `DATA RACE` warning on the target and a clean control. We ran the `ExperimentRace` task with `-count=10`, `-timeout=2m`, and 35 processes (targets + controls). We also included additional integration with SharedFace, PrivateFace, and an external mutex.

3.4 Correctness oracle (digest)

The "digest" is not an ad hoc process hash: it is the SHA-256 of a **canonical encoding** of each worker's observable results.

Figure 3. *Oracle pipeline.*



For `Glyph`, the encoding includes destination rectangle, mask point, advance, `ok`, bounds, stride, and an **immediate deep copy** of `Pix` (before the next call invalidates the documented lifetime). Metrics, advances, bounds, kern, and `DrawString` pixels follow the same typed-tag scheme. The oracle is deterministic because: (1) the font and options are fixed; (2) the serial path does not share scratch across workers; (3) the pixel copy captures the value before reuse. A **digest mismatch** indicates a bit-for-bit divergence from the serial reference result — functional evidence, not merely from the race detector.

3.5 Ownership alternatives (A–F)

ID	Harness implementation	Shared	Exclusive
A	External sync.Mutex around the Face	1× Face	Drawer/Dst/sink; mask consumed under lock
B	opentype.NewFace / Close per Glyph	*sfnt.Font	Face per operation
C	Face created in worker setup	*sfnt.Font	Face per worker (baseline)
D	sync.Pool of Faces	*sfnt.Font + pool	Borrowed Face until Put after consumption
E	Immutable Config + Scratch (Buffer+Rasterizer+mask)	Config / Font	Scratch per worker
F	E + rasterization into caller-owned *image.Alpha	Config / Font	Scratch + caller destination

We validated all alternatives against the sequential oracle and under `-race` (count=5) in the declared pattern.

3.6 Benchmark methodology

Parameter	Value
Tool	testing.B with ReportAllocs
Samples	count=10, benchtime=1s
Workers	W ∈ {1,2,4,8,16}
Common matrix	19 identical BenchmarkAlt* names across A–F
Profiles	CPU, heap, block, mutex (benchtime=2s on contention profiles)
Retention	isolated process, 16 workers, forced GC

3.7 Statistical analysis

Comparisons use pinned `benchstat` (`golang.org/x/perf`, module sum recorded in the manifest). The tool reports Δ% and **p**-value (Mann–Whitney); `~` means **failure to reject the null hypothesis of equality** at the tool's usual level — **not** a formal proof of equivalence. We did not perform TOST in this report; therefore we never conclude "statistical equivalence", only **indistinguishability in the experiments performed**. Where `benchstat` prints `p=0.000`, we report $\mathbf{p < 0.001}$. The practical effect size is Δ% vs C (base = Face-per-worker).

3.8 Reproducibility

We used a digest-pinned image, a versioned runner, raws and SHA-256 of key artifacts, and a harness module with `go.mod`/`go.sum`. See Section 11 (artifact availability).

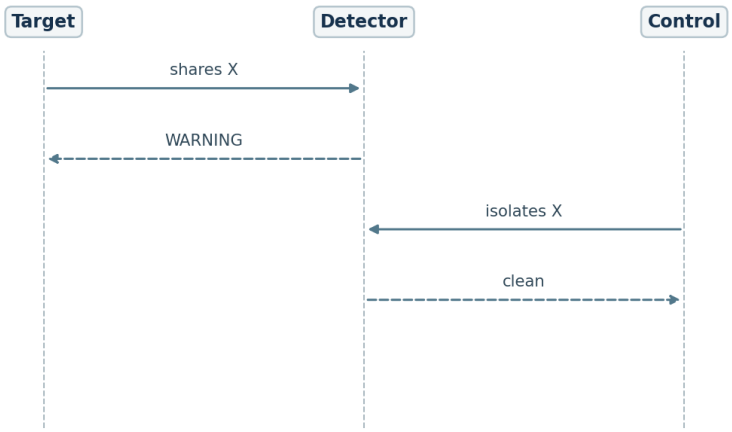
4. Results I — races and functional consequences (RQ1, RQ2)

The non-concurrency contract is established by the documentation [1]. The experiments do not "prove" the normative text; they show that the observed behavior **is consistent with that contract** and that sharing the instance produces real concurrent accesses.

The causal criterion is a **target** that shares domain X and an orthogonal **control** that isolates X. Empirical confirmation = warning on the target and a clean control. We ran 35 processes (`-race`,

count=10). **13/13** IDs showed a racy target and clean controls; SharedFace integration produced warnings; the integration controls (PrivateFace, external mutex) produced **0** warnings.

Figure 4. *Causal target/control criterion.*



4.1 Matrix of the 13 hypotheses

Analytical separation: **Face-specific** (OT-*) vs **adjacent state** on the drawing path (DR-*, P9-*, SF-*). The detector's warning count does **not** measure problem intensity; the `-race` column reports reproducibility. Without `-race` (count=10): 126 digest mismatches, 3 panics, 0 timeouts — failures only on shared targets; controls clean.

Table 1a — Face-specific (`opentype.Face` / scratch / mask)

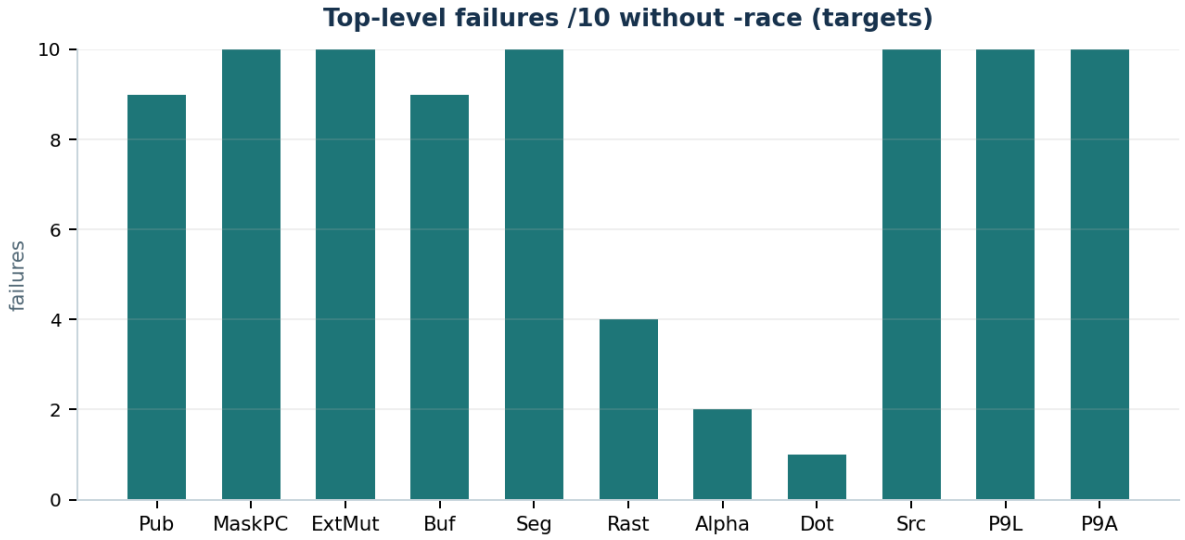
ID	Shared state	Hypothesis	Control	-race	Without -race
OT-001	Cold metrics cache	Two Metrics contend on metricsSet/metrics	Serial prewarm	10/10 · 3 warnings	No functional symptom in this collection
OT-002	sfnt.Buffer	Concurrent views/reslices on scratch	Buffer per worker	10/10 · 68 warnings (2 variants)	Frequent digest mismatch
OT-003	Buffer.segments	Segment alias vs reuse	Deep copy / distinct buffer	10/10 · 3 warnings	Digest mismatch
OT-004	vector.Rasterizer	Overlapping Reset/path/Draw	Rasterizer per worker	10/10 · 16 warnings	Digest + panics (bounds)
OT-005	image.Alpha (producer×producer)	Shared header/Pix	Alpha per worker	10/10 · 3 warnings	Digest mismatch
OT-006	returned mask vs next Glyph	Consumer vs producer	Consume-before-reuse / deep-copy	10/10 · 6 warnings	Digest mismatch (10/10)
OT-007	external mutation of the mask	Caller misuse	Mutate only the copy	10/10 · 20 warnings (2 variants)	Digest mismatch (10/10)

Table 1b — Adjacent state (Drawer / Plan 9 / ReaderAt)

ID	Shared state	Hypothesis	Control	-race	Without -race
DR-001	Drawer.Dot	Shared Draw	Private Drawer / BoundString	10/10 · 2 warnings	Rare symptom (1/10)
DR-002	Shared Dst	Two DrawMask on same backing	Dst per worker	10/10 · 6 warnings	No top-level FAIL in this collection
DR-003	Mutable Src	Writer vs composition	Uniform / private Src	10/10 · 4 warnings	Digest mismatch (10/10)
P9-001	Lazy Plan 9 cache	Concurrent cold range	Prewarmed range	10/10 · 7 warnings	Digest mismatch (10/10)

ID	Shared state	Hypothesis	Control	-race	Without -race
P9-002	Mutated Plan 9 atlas	External atlas mutation	Stable atlas	10/10 · 1 warning	Digest mismatch (10/10)
SF-001	source.minSize (ReaderAt)	Late write after parse	[]byte / preconditioned	10/10 · 3 warnings	No functional symptom in this collection

Figure 5. *Operational evidence without* `-race`*: top-level failures per case over 10 repetitions, only on shared targets.*



Abbrev.	Case
Pub	public SharedFace pairs
MaskPC	mask producer–consumer
ExtMut	external mask mutation
Buf / Seg / Rast / Alpha	shared buffer, segments, rasterizer, Alpha
Dot / Src	Drawer Dot / mutable Src
P9L / P9A	Plan 9 lazy range / atlas

RQ1. On the Face-specific path, Metrics, Buffer, Segments, Rasterizer, and mask require exclusive ownership; adjacent states (Drawer, Dst, Src, Plan 9, ReaderAt) are orthogonal sources that also require exclusivity when shared.
RQ2. The detector identifies concurrent accesses; digest mismatch and panics without `-race` constitute functional consequences on the tested workloads.

5. Results II — ownership patterns (RQ3, RQ4)

ID	Pattern	Idea
A	Mutex	Single Face + lock
B	Face/op	NewFace on every Glyph
C	Face/worker	Exclusive Face per goroutine
D	Pool (sync.Pool)	Get/Put of Faces
E	Scratch	Config + scratch (via public API)
F	Caller dest	E + caller-owned *image.Alpha

All of A–F: identical oracle and clean `-race` count=5 in the declared pattern.

5.1 Geomean definition and statistical analysis

The **common matrix** aggregates the 19 `BenchmarkAlt*` benchmarks present in all A–F packages (same names, `count=10`, `benchtime=1s`, `GOMAXPROCS=4`):

- `AltSerialGlyph/{Repeated,VariedUnicode}`
- `AltConcurrentGlyph/W{1,2,4,8,16}/{Repeated,VariedUnicode}`
- `AltConcurrentDrawString/W{1,2,4,8,16}/ShortASCII`
- `AltAcquireRelease/W{1,4}`

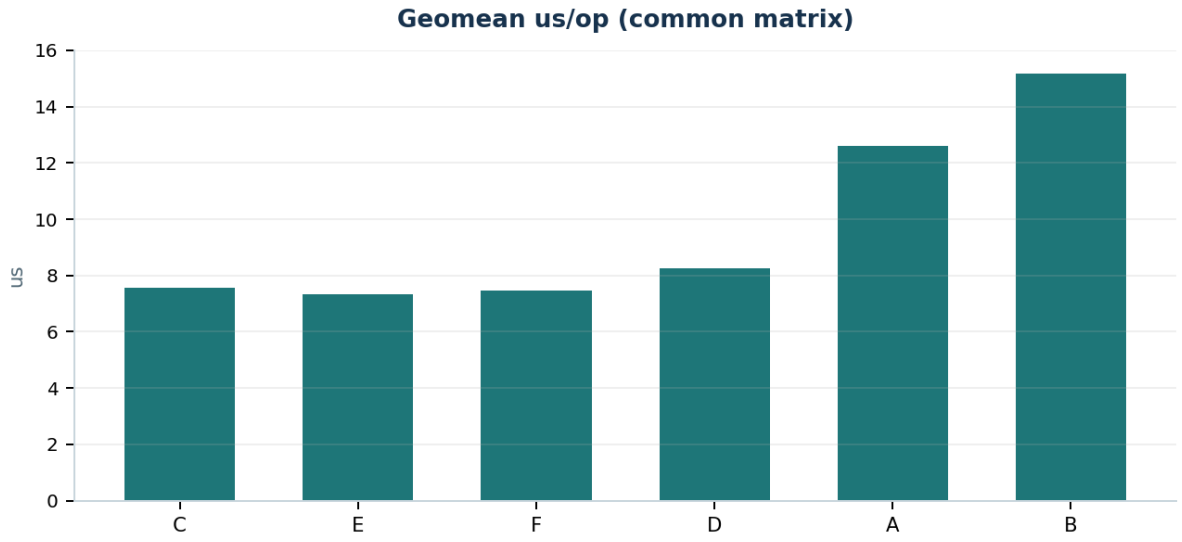
The `benchstat` **geomean** is the geometric mean of the mean times of those points, with equal weight per name. The decision question uses the **ratio vs C**: Δ% and classification `~` / significant.

Table 2. *`benchstat` summary vs C (`GOMAXPROCS=4`). `~` = insufficient evidence of a difference on the glyph path (does not imply equivalence).*

Strategy	Geomean (µs)	Δ vs C	Classification
C	7.570	—	base
E	7.339	−3.05%	~ on Glyph
F	7.446	−1.64%	~ on Glyph
D	8.262	**+9.14%**	significantly slower
A	12.62	**+66.76%**	significantly slower
B	15.19	**+100.71%**	significantly slower

Illustrative points (W4/Repeated): C 2.755 µs ±10%; E `~` (p=0.404); F `~` (p=0.256); D +20.05% (p < 0.001); A +213.38% (p < 0.001); B +200.60% (p < 0.001).

Figure 6. *Common-matrix geomean (µs/op). Descriptive ordering C ≈ E ≈ F < D < A < B; the decision uses Table 2, not a single-point ranking.*



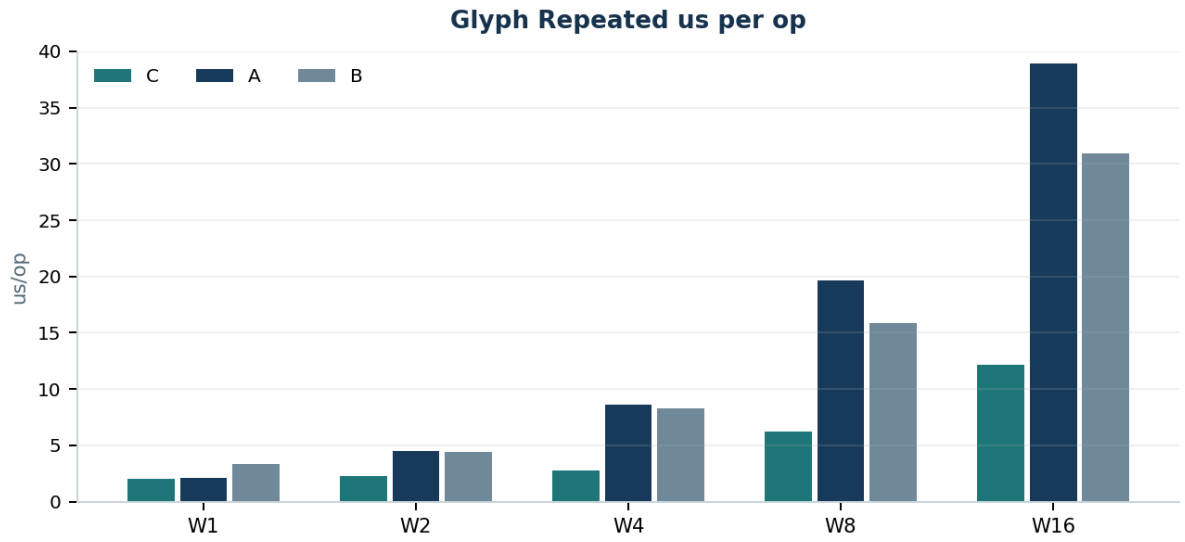
Under the geomean metric and the `benchstat` classification, **C, E, and F were statistically indistinguishable in the experiments performed** on the Glyph path. C remains the preferable option **among those available** because it already satisfies the contract **without** a new capability or API change. The E and F results do not provide evidence of a speed gain that would justify a patch.

5.2 Scalability

In the charts below, **W** is the number of goroutines that run `Glyph` in parallel. **µs/op** is the mean time per operation from `testing.B`.

AcquireRelease isolates the cost of obtaining/releasing ownership (Lock/Unlock, NewFace/Close, no-op in C, Get/Put in D).

Figure 7. *`Glyph` latency (Repeated) as a function of W. Bars in order C, A, B.*



Order	Alternative	Reading
1	C Face/worker	Scales up to ~the container parallelism quota (4 logical CPUs); oversubscription (W8, W16) increases latency
2	A mutex	Nearly equal to C at W1; latency grows sharply from W2 onward
3	B Face/op	High cost at all evaluated W levels

C numbers for Repeated: W4 = 2.755 µs; W8 = 6.237 µs; W16 = 12.19 µs — saturation coherent with the 4-logical-CPU quota, not unbounded linear scaling. A's latency goes from 2.126 µs/op at W1 to 38.97 µs/op at W16.

Figure 8. *W4 Repeated for all alternatives (µs/op). E/F indistinguishable from C (~); D +20%; A/B ~3×.*

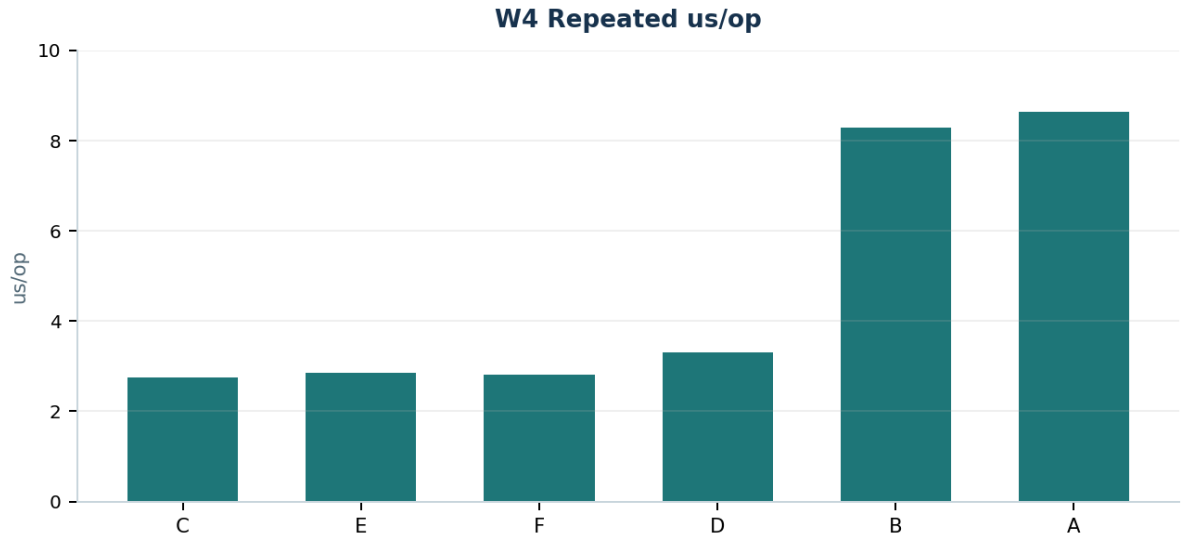
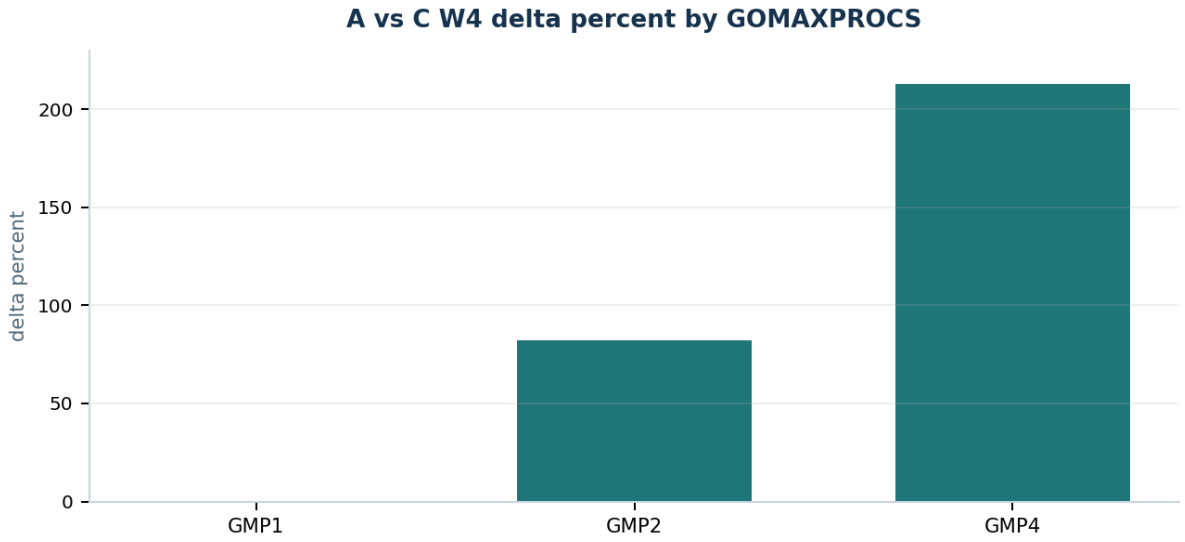


Figure 9. *Mutex contention under the quota's effective parallelism: at `GOMAXPROCS=1`, A approaches C; at 4P, A is +213% at W4.*



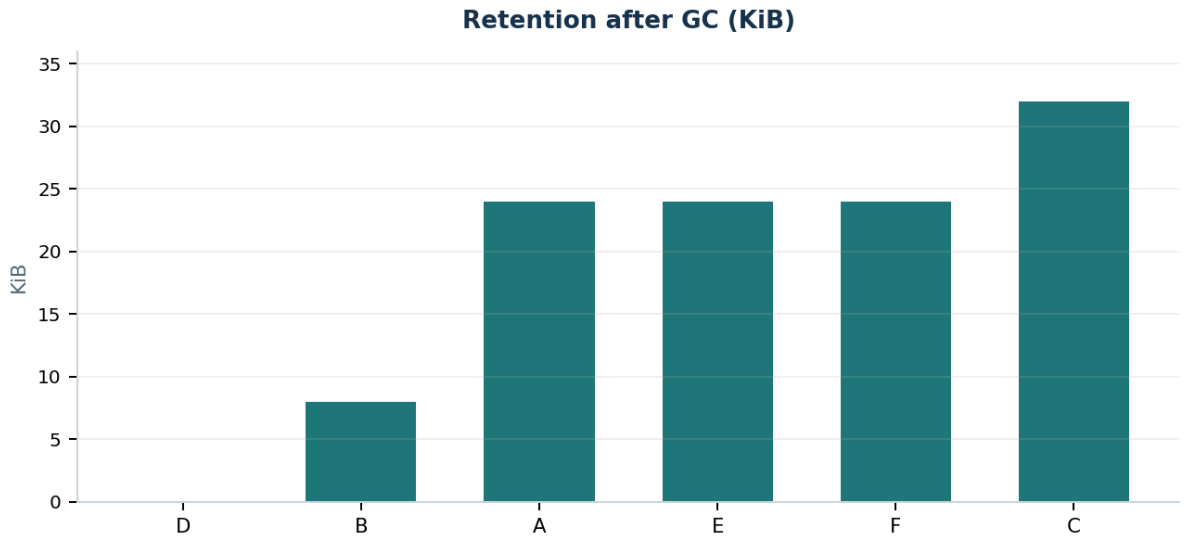
Mutex profile (`runtime/pprof` mutex profile, W4, `benchtime=2s`): the metric is the **cumulative mutex contention weight** (sampled delay attributed to locks), **not** the benchmark's wall-clock duration. C **8.96 ms** · A **8.04 s** — three orders of magnitude in sampled contention, coherent with the serialization introduced by A.

5.3 Allocation, retention, and `sync.Pool` (D)

	C	E	F	D	A	B
Δ geomean vs C	—	−3% (~)	−2% (~)	**+9%**	**+67%**	**+101%**
AcquireRelease/W1	93 ns	86 ns	87 ns	111 ns	90 ns	**703 ns**
AcquireRelease/W4	137 ns	137 ns	122 ns	174 ns	**446 ns**	**2511 ns**
SerialGlyph B/op	80	80	80	80	80	**3184**
SerialGlyph allocs	1	1	1	1	1	**9**

F with a reused destination: **0 B/op**. B: +800% allocs.

Figure 10. *Memory trade-off (16 workers, forced GC): attributable retention after GC.*



D uses `sync.Pool`. The collector may empty the pool between cycles: **0 retention after forced GC is not a net memory benefit** — it is a trade-off between retention and reacquisition. In the same test: mallocs E/F ~9.8k · C 10.2k · B 19k · **D 47k** (~4.6× C). Zero retention appears together with ~4.6× more mallocs; interpreting "0 KiB" in isolation would be misleading.

5.4 Lifetime (RQ4)

Figure 11. *Cost × mask lifetime.*

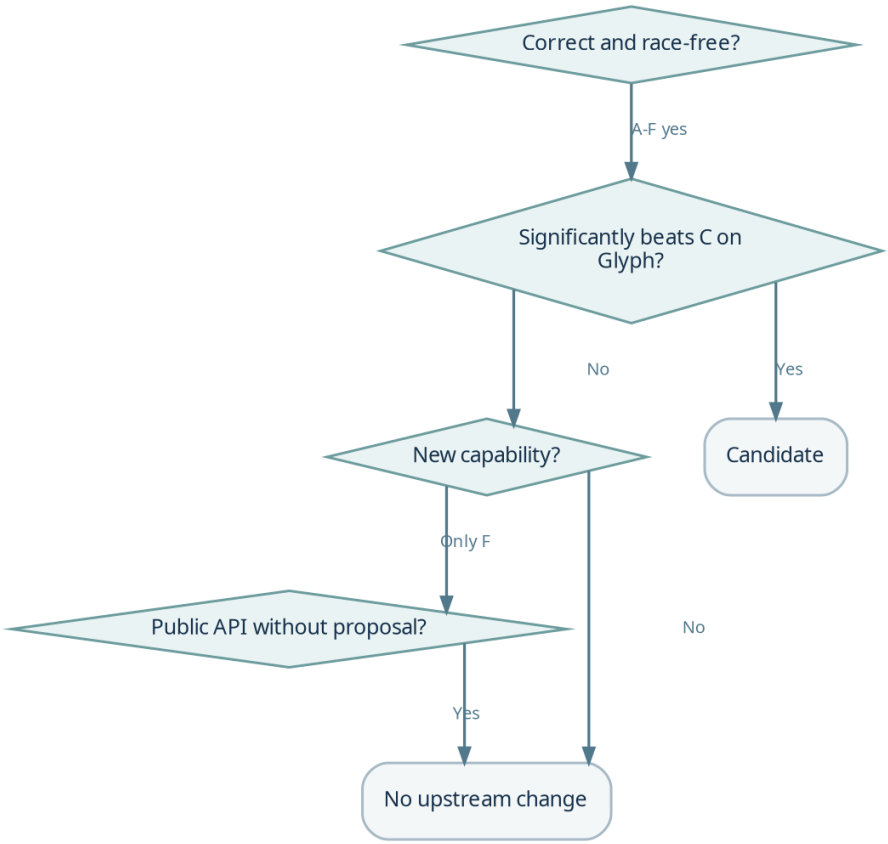


C and E are fast with short lifetime (current contract; E already via the low-level public API). F obtains independent lifetime in C's cost band, but utility depends on a **new public promise**. B isolates lifetime at high cost. A serializes without extending lifetime.

RQ3. Face-per-worker (C) outperforms A, B, and D on the evaluated workloads; after available-quota saturation, C remains the lowest-cost pattern among those options; C, E, and F were indistinguishable (~) on Glyph.
RQ4. Among the requirements and alternatives evaluated, only independent mask lifetime constitutes a missing capability; it would require a public API change (F).

6. Decision: no upstream change justified

Figure 12. *Architectural gate.*



No alternative beat C in a statistically significant way on Glyph. C is already supported by the current API (zero diff). F remains gated by a proposal. Test/Race of packages `font` / `opentype` / `sfnt` PASS; upstream diff empty.

7. Recommendations

1. Share `*sfnt.Font` from immutable `[]byte`.
2. **One Face per worker.**
3. Consume/copy the mask before the next `Glyph` on the same Face.
4. Private Drawer/Dot/Dst; stable Src.
5. Lifetime without a new API: `Font` + `Buffer` + `Rasterizer` + own Alpha.

8. Limitations and threats to validity

Construct validity

The experiment measures concurrent accesses (race detector), bit-for-bit divergence from the serial oracle (digest), and latency/allocation under synthetic harness workloads. Warning counts are not a severity metric; scheduling affects reports. Retention and malloc results depend on the collector and on `sync.Pool`. Target/control designs and the SHA-256 oracle reduce the risk of confusing a detector artifact with a functional failure.

Internal validity

Alternative explanations for performance differences include saturation of the 4-logical-CPU quota, oversubscription at W8/W16, and acquire/release cost, not only the ownership pattern. We interpret scalability against the quota available to the process, not against a pinned physical topology. The `benchstat ~` classification indicates failure to reject equality, not formal equivalence; there is no TOST and no pre-registered equivalence interval.

External validity

We used a single font fixture (Go Regular), one size/DPI/hinting configuration (12 / 72 / HintingNone), only `linux/amd64` with Go 1.25.0, and one host under a container with a 4-logical-CPU / 4 GiB quota, without cross-machine validation. Workloads are synthetic (harness `Glyph`/`DrawString`), without shaping of real text; the glyph distribution may differ from UI/production. Face-specific findings do not automatically generalize to `basicfont`; Plan 9 and ReaderAt are adjacent states. Misuse in real software and a multi-font/multi-host matrix remain future work.

Conclusion validity

Absolute times depend on the host; the descriptive ordering and `benchstat` classifications are the signal we treat as stable. With `count=10` and Mann–Whitney via `benchstat`, conclusions of significant difference (A, B, D vs C) and of indistinguishability (`~` for E, F) are limited to the experiments performed. To strengthen RQ3 beyond a technical report, a useful minimal matrix would be 3 fonts × 2 sizes × 2 hosts, testing whether the ordering C/E/F < D < A/B holds — without combinatorial explosion.

9. Related work

9.1 History and contract of the Go API

The `font.Face` contract and mask lifetime come from the design accepted in 2015 [1]. The `sfnt` / `opentype` split (CL 73870, issue #22451) and the implementation with reused scratch (CL 255237) preserve that model [3]. `sfnt.Buffer` and the lifetime of `Segments` were made explicit in later CLs [4]. In the recorded searches, we found no accepted proposal that makes `font.Face` concurrent.

9.2 Data-race detection and the memory model

The Go race detector [5], grounded in ThreadSanitizer / happens-before [10, 11], and the Go memory model [6] delimit what a warning means: concurrent access without happens-before synchronization, not necessarily a "concurrent-contract bug" if the API already forbids sharing. We use the detector as a causal instrument (target vs control), not as an isolated product-defect verdict.

9.3 Empirical studies of concurrency in Go

Tu et al. [16] characterize concurrency bugs in production Go software and show that shared memory remains an important defect source. Chabbi and Ramanathan [17] study real data races in Go programs. Our focus is complementary: instead of mining bugs in applications, we characterize the **ownership boundary** of a library API and the consequences of violating it.

9.4 Ownership, aliasing, and concurrent API design

Ownership and aliasing models [20–22] and Rust's borrowing discipline [20] inspire the **share** / **scratch** / **borrow** separation without requiring a type system. Work on concurrency APIs and contracts [23] reinforces that normative documentation combined with reused mutable state is a common — and fragile — pattern under parallel use.

9.5 Benchmark methodology and negative results

Rigorous performance evaluation requires multiple samples and statistical tests [24, 25]; `testing.B` [7] and `benchstat` [8] operationalize that in the Go ecosystem. Negative results and evidence-based performance engineering [26, 27] frame the decision **not** to change upstream: absence of a significant gain under declared requirements is a publishable result, not an experimental failure.

10. Conclusions

We empirically characterized the API's ownership boundary, observed functional consequences of violating that boundary, evaluated six alternative strategies, and found no evidence that any upstream modification presents sufficient advantage under the studied requirements.

RQ	Answer
RQ1	On the Face-specific path, Metrics, Buffer, Segments, Rasterizer, and mask require exclusive ownership; Drawer/Dst/Src/Plan9/ReaderAt are adjacent states with the same discipline (Tables 1a–1b).
RQ2	Sharing produces observable corruption and panics on the tested workloads, in addition to detector warnings (Section 4; Figure 5).
RQ3	Face-per-worker (C) outperforms A/B/D on the evaluated workloads; scales up to the available parallelism quota; C/E/F were indistinguishable (~) on Glyph (Table 2; Figures 6–9).
RQ4	Among the requirements and alternatives evaluated, only independent mask lifetime is a missing capability; it would require a public API (F) (Figure 11).

Among the requirements and alternatives evaluated, we found no evidence that justifies an upstream change (Figure 12).

11. Artifact availability

Item	Value
Container digest	sha256:341bc773b805e466baf06866246a8eaa41acd2da5e91b5a191b7053ece887912
Baseline commit	3ebddc7c54bd879f8d84d11db82892726f5192fd (golang.org/x/image v0.45.0)
Upstream license	BSD-style (Go)

The reproduction package (pinned image, runner, race and benchmark raws, `benchstat` outputs, and oracle) will be made available with the public archive. Reproduction goal: from the pinned image, re-run `ExperimentRace` / `ExperimentBenchmark` / `Benchstat` and regenerate Tables 1–2 and the latency figures.

References

Go implementation and history

1. Nigel Tao et al. CL 13463 — **shiny/font: new package for drawing text on an image.** Gerrit, 2015. https://go-review.googlesource.com/c/exp/+/13463.
2. Go Proposal Process. https://github.com/golang/proposal.
3. golang/go#22451; CL 73870; CL 255237 — `font/opentype` lineage over `sfnt`.
4. CL 34532 / CL 256957 — `sfnt.Buffer` privacy and `Segments` lifetime.
5. The Go Blog — **Introducing the Go Race Detector.** https://go.dev/blog/race-detector.
6. The Go Memory Model. https://go.dev/ref/mem.
7. Package `testing` — benchmarks. https://pkg.go.dev/testing.
8. `golang.org/x/perf/cmd/benchstat`. https://pkg.go.dev/golang.org/x/perf/cmd/benchstat.
9. Documentation for `sfnt.Font` / `opentype.Face` in `golang.org/x/image v0.45.0`.

Race detection and concurrency

1. K. Serebryany and T. Iskhodzhanov. **ThreadSanitizer — data race detection in practice.** WBIA, 2009.
2. C. Flanagan and S. N. Freund. **FastTrack: efficient and precise dynamic race detection.** PLDI, 2009.
3. S. Savage et al. **Eraser: a dynamic data race detector for multithreaded programs.** ACM TOCS, 1997.
4. R. O'Callahan and J.-D. Choi. **Hybrid dynamic data race detection.** PPoPP, 2003.
5. D. Marino, M. Musuvathi, and S. Narayanasamy. **LiteRace: effective sampling for lightweight data-race detection.** PLDI, 2009.
6. P. Pratikakis, J. S. Foster, and M. Hicks. **LOCKSMITH: context-sensitive correlation analysis for race detection.** PLDI, 2006.

Empirical studies of concurrency in Go

1. T. Tu, X. Liu, L. Song, and Y. Zhang. **Understanding Real-World Concurrency Bugs in Go.** ASPLOS, 2019.
2. M. Chabbi and M. K. Ramanathan. **A Study of Real-World Data Races in Golang.** PLDI, 2022.
3. Z. Liu, S. Xia, Y. Liang, L. Song, and H. Hu. **Who Goes First? Detecting Go Concurrency Bugs via Message Reordering.** ASPLOS, 2022.
4. N. Dilley and J. Lange. **Bounded verification of message-passing concurrency in Go using Promela and Spin.** HATRA / PLACES, 2020.

Ownership, aliasing, and APIs

1. R. Jung et al. **RustBelt: securing the foundations of the Rust programming language.** POPL, 2018.
2. J. Boyland. **Alias burying: unique variables without destructive reads.** Software: Practice and Experience, 2001.
3. D. Clarke, J. Noble, and J. Vitek (eds.). **Aliasing in Object-Oriented Programming.** Springer LNCS, 2013.
4. S. Lu, S. Park, E. Seo, and Y. Zhou. **Learning from mistakes — a comprehensive study on real world concurrency bug characteristics.** ASPLOS, 2008.

Benchmark methodology and negative results

1. A. Georges, D. Buytaert, and L. Eeckhout. **Statistically rigorous Java performance evaluation.** OOPSLA, 2007.
2. T. Kalibera and R. Jones. **Rigorous benchmarking in reasonable time.** ISMM, 2013.
3. S. Hanenberg. **Faith, hope, and love: an essay on software science's neglect of human factors.** OOPSLA, 2010 (context of empirical evidence and claims).
4. C. Curtsinger and E. D. Berger. **Stabilizer: statistically sound performance evaluation.** ASPLOS, 2013.

Appendix — List of figures and tables

Fig.	Type	Content
1	Flow	Ownership (structural / scratch / borrowed)
2	Flow	Method
2b	Flow	SHA-256 oracle
3	Sequence	Target/control
4	Bars	Top-level failures without -race
5	Bars	Common-matrix geomean

Fig.	Type	Content
6	Grouped bars	C vs A vs B by workers (W)
7	Bars	W4 in µs/op
8	Bars	A × GOMAXPROCS
9	Bars	Retention after GC
10	Flow	Cost × lifetime
11	Flow	Decision

Tab.	Content
1a–1b	13 hypotheses (Face-specific vs adjacent)
2	benchstat vs C (geomean and classification)